

FoodExpress

Piattaforma di food delivery con gamification

Oggetto del progetto — applicazione web dove l'utente sfoglia il menu, ordina piatti e li riceve a domicilio. A ogni ordine guadagna punti fedelta, sale di livello e sblocca badge; una classifica premia i clienti migliori. Il sistema stima distanza, costo e tempo di consegna tramite un servizio esterno di geocoding.

Angular 18 · Spring Boot 3 · MySQL · REST + JWT

Distribuzione del team

Progetto individuale — ruoli ricoperti a rotazione secondo la metodologia del corso

1

Team Leader

Pianifica le attività e le distribuisce sulla board.

Colonna: TO DO

2

Developer

Sviluppa le funzionalità ed esegue un primo test.

Colonna: DONE

3

Tester

Verifica che i requisiti siano soddisfatti.

Colonna: VERIFIED

Evidenze delle cerimonie

Pianificazione, avanzamento e verifica gestiti su Trello

Flusso di lavoro (board Trello)

- Pianificazione iniziale: scomposizione in task e stima.
- Avanzamento: i task passano TO DO → DOING → DONE.
- Verifica: il Tester sposta i task in VERIFIED.
- Ogni card riporta gli Story Points assegnati.

Story Points

~34

Story Points totali

- Pivot: CRUD semplice = 2 SP
- Scala di Fibonacci (1,2,3,5,8,13)

Disegno funzionale (Miro)

Macro-funzioni ed entita del dominio

3 Macro-funzioni

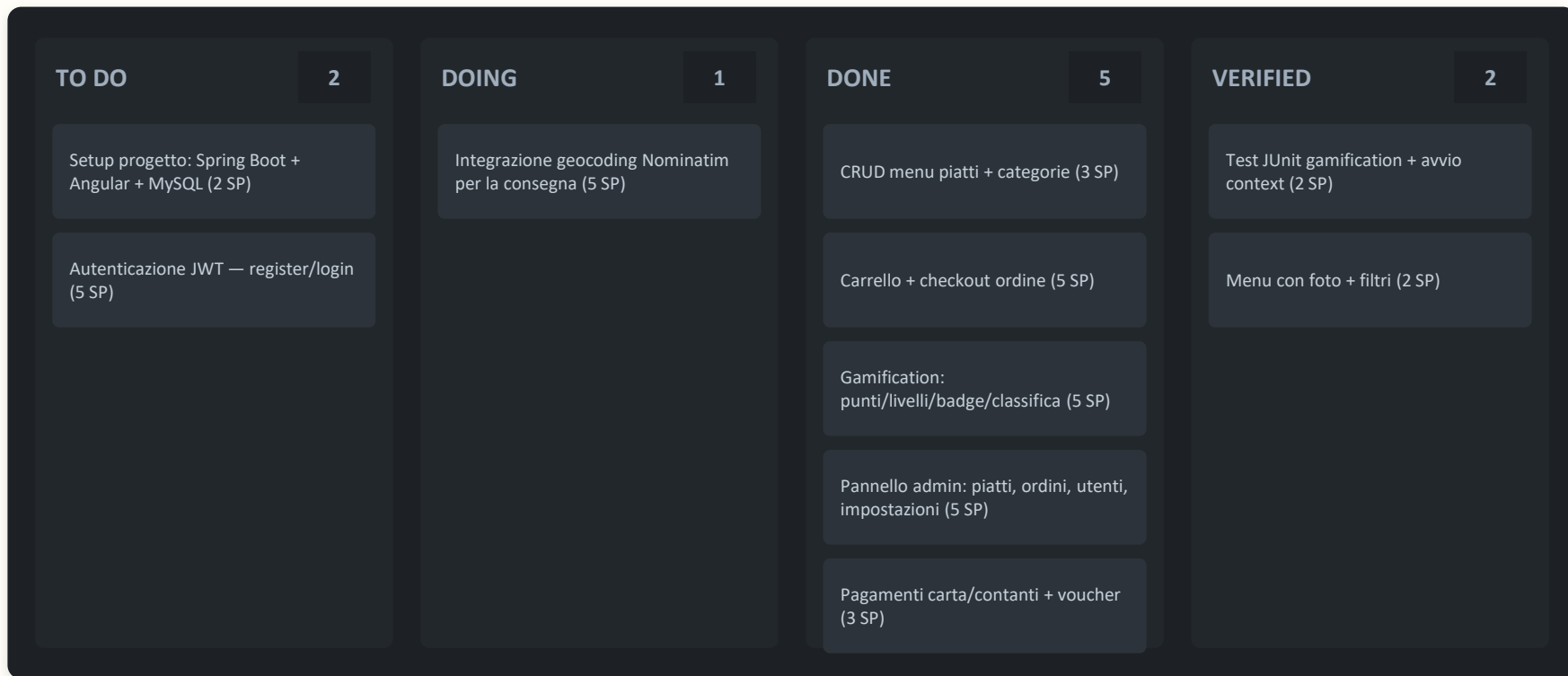
- Catalogo & Menu (piatti, categorie)
- Carrello & Ordini + geocoding consegna
- Gamification (punti, livelli, badge, classifica)

Entita

Utente · Categoria · Piatto · Carrello · VoceCarrello ·
Ordine · VoceOrdine · Badge · UtenteBadge · Voucher ·
Impostazioni

Board Trello

Colonne TO DO / DOING / DONE / VERIFIED · board pubblica trello.com/b/N9uHwykS · ~34 Story Points



Test cases

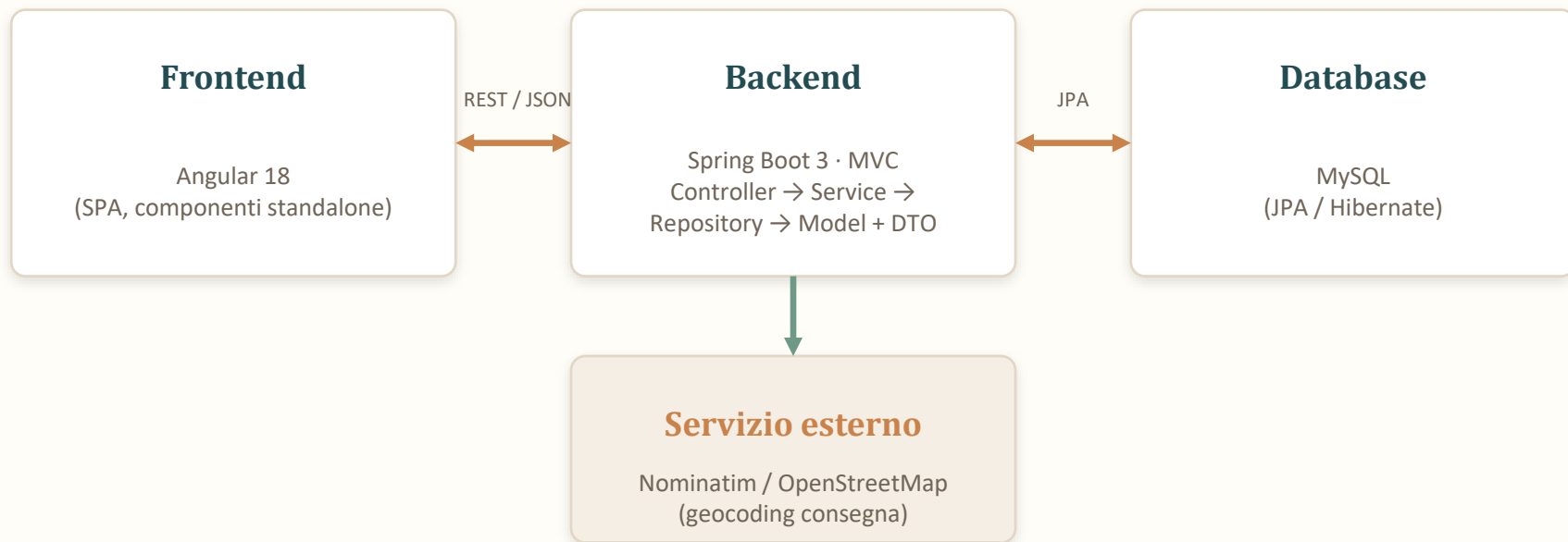
Test automatici JUnit + smoke test del context Spring

Test	Risultato atteso
Calcolo livello dai punti	livello = 1 + punti/100 (0→1, 99→1, 100→2)
Completa ordine (assegnaPunti)	punti aggiornati, livello ricalcolato, badge sbloccato
Badge gia posseduto	non viene sbloccato di nuovo
contextLoads (H2)	l'intero context Spring si avvia correttamente

Esito: **BUILD SUCCESSFUL** (gradlew test)

Disegno architetturale

Tre layer separati, comunicazione REST, pattern MVC



Sicurezza: autenticazione JWT stateless · Password cifrate con BCrypt

Funzionalità implementate

Lato utente/cliente e lato amministratore

Cliente

- Registrazione / login (JWT)
- Menu con filtri e foto dei piatti
- Carrello + checkout come ospite o registrato
- Stima di consegna (geocoding)
- Pagamento carta/contanti + voucher sconto
- Punti fedeltà, livelli, badge, classifica
- Modifica del proprio profilo

Amministratore

- Gestione menu: CRUD piatti + foto + ricerca
- Attiva/disattiva disponibilità piatto
- Gestione ordini: accetta/rifiuta/stato spedizione
- Modifica indirizzo di consegna di un ordine
- Gestione utenti: blocca / elimina / vedi ordini
- Impostazioni sito: nome, logo, contatti, news

Dettaglio: Checkout con stima di consegna

Integrazione REST esterna + logica di gamification

1

L'utente inserisce l'indirizzo e conferma l'ordine.

2

GET /api/ordini/stima → il backend chiama Nominatim, geocodifica l'indirizzo e calcola la distanza dal ristorante (formula haversine) → costo e tempo di consegna.

3

POST /api/ordini/checkout → crea l'ordine (snapshot delle voci), applica l'eventuale voucher, salva metodo di pagamento.

4

Se l'utente è registrato: assegna i punti fedeltà, ricalcola il livello e sblocca i badge.

Componenti

OrdineService.checkout()

orchestrazione ordine + punti

GeocodingService

WebClient → Nominatim + haversine + fallback

GamificationService

punti, livelli, badge

Feedback dell'esperienza

Cosa ho imparato

Separazione frontend/backend, API REST e pattern MVC, gestione dello stato lato client, integrazione di un servizio esterno e progettazione di meccaniche di gamification.

Difficolta affrontate

Evoluzione dello schema DB (colonne ed enum), servizio dei file immagine, e affidabilita del servizio di geocoding (gestita con un fallback).

Sviluppi futuri

Pagamenti reali, tracking live della consegna, recensioni dei piatti e notifiche.